

Validated Computing

Objectives:

- fast computations
- flexible computations
- track all sources of error.

Why?

- used floating point comp. involve errors: there is some rounding involved **computations are not exact.**

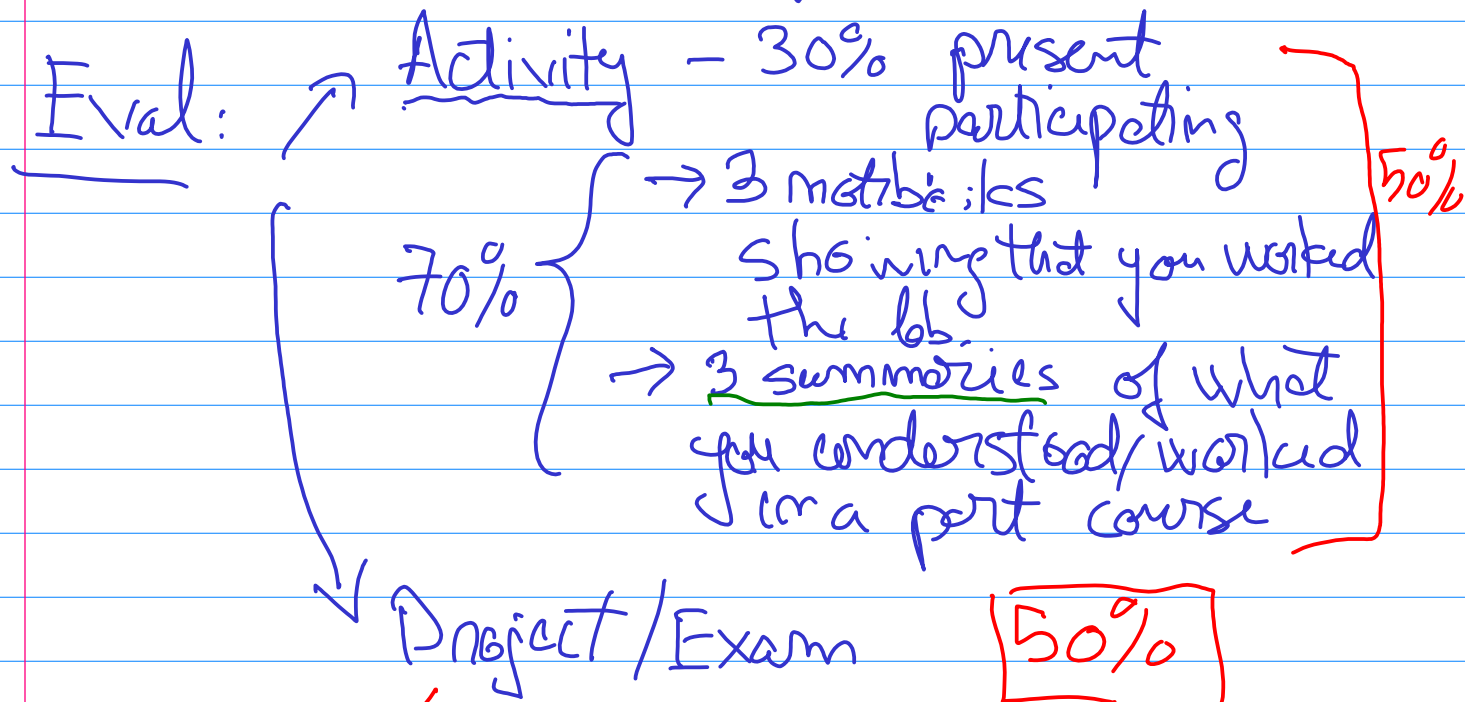
- usually the error is small

float32 $\sim$ 6 significant digits
float64 $\sim$ 16 sig. digits.

- if you do repeated computations the errors accumulate and may become Big.

What can we do to control the errors?

Structure :- mix lab and course
- Notebooks in Python
(VS code, Google Colab, next time)



Subjects before Christmas.

Controlled errors: importance

- critical applications: nuclear reactor, military, engines in cars.

Applications:

- proving math results with computer assisted methods.
- AI tools: improve reasoning verification abilities, decrease hallucinations

Motivation:

Exact computing with rational numbers
→ Symbolic Computing.

Rational number $q = \frac{a}{b}$, $a, b \in \mathbb{Z}$
 $b \neq 0$

Pari-GP: quick Symbolic Computations
(fmu)

$$S_2 = 1 + \frac{1}{2}$$

$$S_3 = 1 + \frac{1}{2} + \frac{1}{3}$$

$\vdots$

$$S_{100} = 1 + \frac{1}{2} + \dots + \frac{1}{100} = \frac{\overbrace{40 \text{ digit}}^{132 \text{ bits}}}{\underbrace{40 \text{ digit.}}_{132 \text{ bits}}}$$

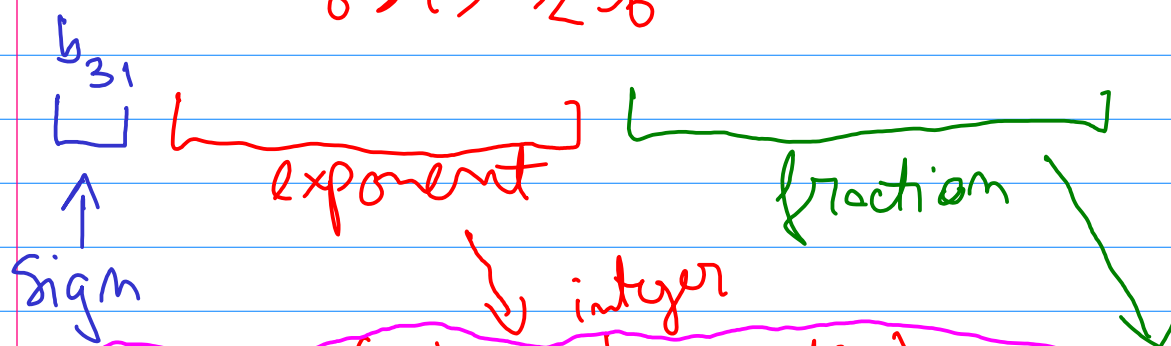
$$S_{1000} = 1 + \frac{1}{2} + \dots + \frac{1}{1000} = \frac{1440 \text{ bits}}{1440 \text{ bits}}$$

Memory usage is too demanding for
exact rational computations in real life appli.

This was one reason for introducing floating point numbers!

For a given precision $\rightarrow$ a finite but rich set of rational numbers

Float 32: 32 places to put 0 or 1
8 bits $\rightarrow$ 256



$$(-1)^{b_{31}} \times 2^{(b_{30} b_{29} \dots b_{23} - 127)} \times 1.b_{22} b_{21} \dots b_0$$

$b_{31} = 0 \rightarrow +$

$b_{31} = 1 \rightarrow -$

In base 2

$$0.00101 = 1.01 \times 2^{-3}$$

in the fractional part we SAVE a bit by assuming 1st digit is 1.

[Q₁] Smallest positive number in float 32?

$$(-1)^{b_{21}} \times 2^{(E-127)} \times 1.\underbrace{\dots}_{23 \text{ digits}}$$

$$\rightarrow b_{21} = 0 \quad (\oplus)$$

$$\rightarrow \underline{E \text{ smallest}}: E = 0$$

$$\rightarrow \text{smallest fraction?}: 1.0 \dots 0$$

$$\Rightarrow X = 1 \cdot 2^{-127} \cdot 1 = 2^{-127}$$

[Q₂] Biggest number?

$$\rightarrow b_{21} = 0, E = 255, 1.11 \dots 1$$

$$\leadsto 2^{128} \cdot \underbrace{1.11\dots1}_{\sim 2} = 2^{129}$$

Big range of numbers: but they form a finite set



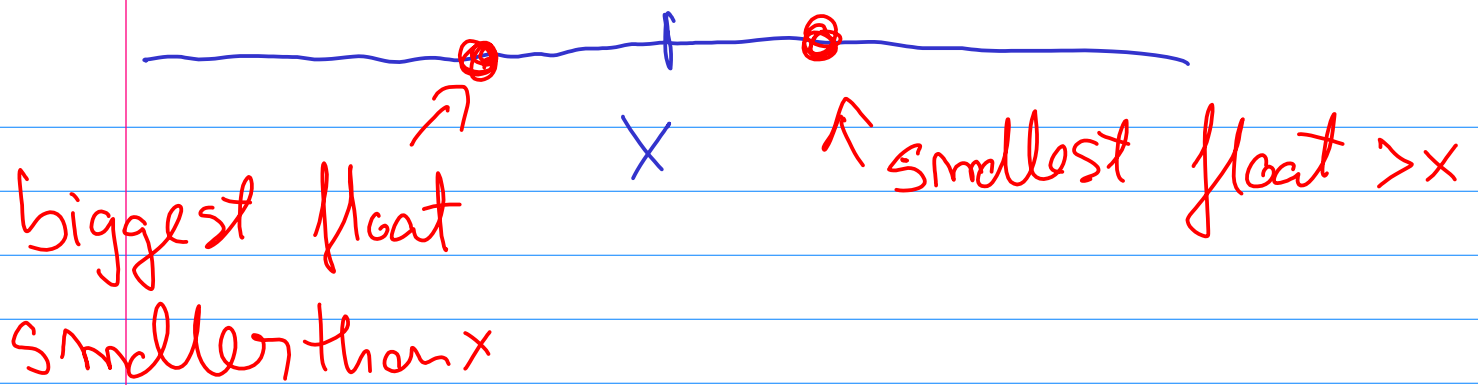
Encodings for $\pm$ infinity

• Not a number.

Floating point numbers do not cover every point on the real line.

If $x \in \mathbb{R}$, not representable as a float:
 - x smaller than all floats
 bigger

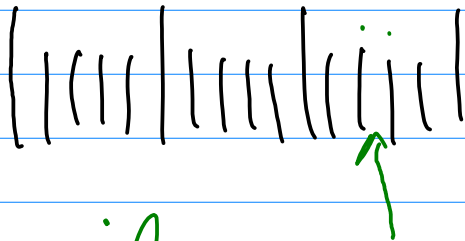
Otherwise: x falls between 2 floating point numbers.



float $v = x;$

We need to make a ROUNDING choice

- upwards / downwards
- closest neighbor.



- if we work in fixed precision we need rounding

- we can think of ways to improve prec when necessary: increasing prec $\rightarrow$ larger mem usage and larger computational cost.

Performing an operation often places us outside the floating point system
→ a rounding takes place → there is an over approximating

ROUND OFF ERROR

$$\begin{array}{r} 1.00 \dots 0 \\ 1.00 \dots 1 \\ \hline \end{array}$$

Significant digits.

→ digits that "fit" inside your floating point representation.

Fact: Floating point numbers always have a denominator which is a power of 2.

$$\frac{m}{2^k}$$

$\frac{1}{10} = \frac{1}{2.5} \rightarrow$ cannot be represented exactly in floating point binary.

Anecdote: Patriot missile failure due to roundoff errors.

24 bit arithmetic

→ measure time in 10ths of a second.

→ $(1/10 \rightarrow \text{binary}) \times \text{number}$

↓
counts 10ths of second
since device started

time since machine started
→ used in model prediction for enemy
missile detection

→

Instead of $1/10$ put $1/8$, $1/16$